



Técnico en ingeniería en computación

ASIGNATURA: Diseño de Software Multiplataforma DPS941

Docente: Ing. Alexander Siguenza

Integrantes:

Carlos Alberto Cornejo Calderón CC251978

David Antonio Leiva Martínez LM160828

Moisés David García Casco GC251462

Alcyr Alexander Figueroa Landaverde FL253210

Nelson Mauricio Solano Guardado SG240238

Grupo: DamagePerSecond – T008

Proyecto Fase II

[GitHub](#)

1. Introducción

El presente documento reúne el trabajo general realizado durante la Etapa 2 del proyecto web de Recursos Humanos. La aplicación busca centralizar procesos administrativos relacionados con empleados, permisos, vacaciones, constancias, solicitudes, aprobaciones y reportes dentro de una misma plataforma.

Durante esta fase se desarrolló una base funcional utilizando React, Next.js y TypeScript. El sistema incorpora autenticación, manejo de roles, dashboard, gestión de solicitudes, filtros, formularios, reportes, generación de constancias en PDF, servicios de Firebase y una integración REST para solicitudes.

El proyecto contempla operaciones para las empresas Didelco, Steel, EFL, DCA, Copro y Propultran. La solución fue desarrollada de forma colaborativa utilizando Git y GitHub, con trabajo distribuido entre los integrantes del equipo.

2. Objetivos

2.1 Objetivo general

Desarrollar la base funcional de un sistema web de Recursos Humanos que permita centralizar procesos administrativos y facilitar la interacción entre empleados, jefaturas y personal responsable de la gestión de solicitudes.

2.2 Objetivos específicos

- Implementar autenticación y control de acceso mediante roles.
- Centralizar solicitudes de permisos, vacaciones y constancias laborales.
- Facilitar el seguimiento de estados y procesos de aprobación.
- Presentar indicadores e información relevante mediante un dashboard.
- Incorporar reportes y generación de documentos en formato PDF.
- Integrar servicios de datos y una API REST para demostrar el flujo cliente-servidor.
- Mantener el proyecto bajo control de versiones y prepararlo para un despliegue público.

3. Alcance del sistema

La Etapa 2 se concentra en la construcción de la base web del sistema y en la implementación de los módulos principales definidos para el proyecto. Las funcionalidades mostradas en este documento corresponden a la versión trabajada y probada por el equipo durante esta fase.

Área	Alcance desarrollado
Autenticación	Ingreso al sistema y cierre de sesión.
Roles	Acceso diferenciado según el tipo de usuario.
Dashboard	Indicadores, gráficas y resumen general.
Solicitudes	Consulta, detalle, filtros y formulario de nueva solicitud.
Reportes	Vista general de información de empleados y exportación.
Constancias	Generación de documento PDF con código QR.
Datos	Servicios Firebase/Firestore y datos de prueba según el módulo.
API REST	Endpoint de solicitudes con GET y POST, validación y consumo mediante Fetch.
Colaboración	GitHub, ramas y commits por integrante.
Despliegue	Publicación funcional en Vercel mediante una URL pública.

4. Tecnologías utilizadas

Tecnología	Uso en el proyecto
Next.js 16	Framework principal y enrutamiento de la aplicación.
React 19	Construcción de componentes e interfaces.
TypeScript	Tipado y organización del código.
Tailwind CSS	Estilos y diseño responsivo.
Firebase Authentication	Autenticación de usuarios.
Cloud Firestore	Servicios de almacenamiento y consulta de datos.
Next.js Route Handlers	Implementación del endpoint REST.
Fetch API	Consumo de la API desde el frontend.
Recharts	Gráficas del dashboard.
pdf-lib	Generación de constancias PDF.
QR Code	Código QR incorporado a constancias.
Git / GitHub	Control de versiones y colaboración.
Vercel	Plataforma utilizada para el despliegue público de la aplicación.

5. Arquitectura y organización del proyecto

El proyecto se organiza de forma modular para separar interfaz, lógica, servicios, tipos y datos de prueba. Entre los directorios principales se encuentran:

```
app/  
components/  
context/  
lib/  
mocks/  
public/  
services/  
types/
```

La carpeta app contiene las rutas y páginas de Next.js; components agrupa componentes reutilizables; context administra estados compartidos; services concentra operaciones relacionadas con datos; mocks contiene información de prueba; y types mantiene las definiciones utilizadas por TypeScript.

Esta separación facilita el mantenimiento del proyecto y permite que diferentes integrantes trabajen sobre módulos específicos sin concentrar toda la lógica en un único archivo.

6. Empresas contempladas

El sistema fue planteado para centralizar procesos de Recursos Humanos asociados a las siguientes empresas:

- Didelco
- Steel
- EFL
- DCA
- Copro
- Propultran

7. Autenticación, sesión y roles

El sistema incorpora autenticación para controlar el acceso a las funcionalidades. Una vez autenticado, el usuario puede visualizar las opciones correspondientes a su rol.

Los roles contemplados en el proyecto son Administrador TI, Gerente, Analista de Nómina, Asistente de Planilla, Jefe Inmediato y Empleado.

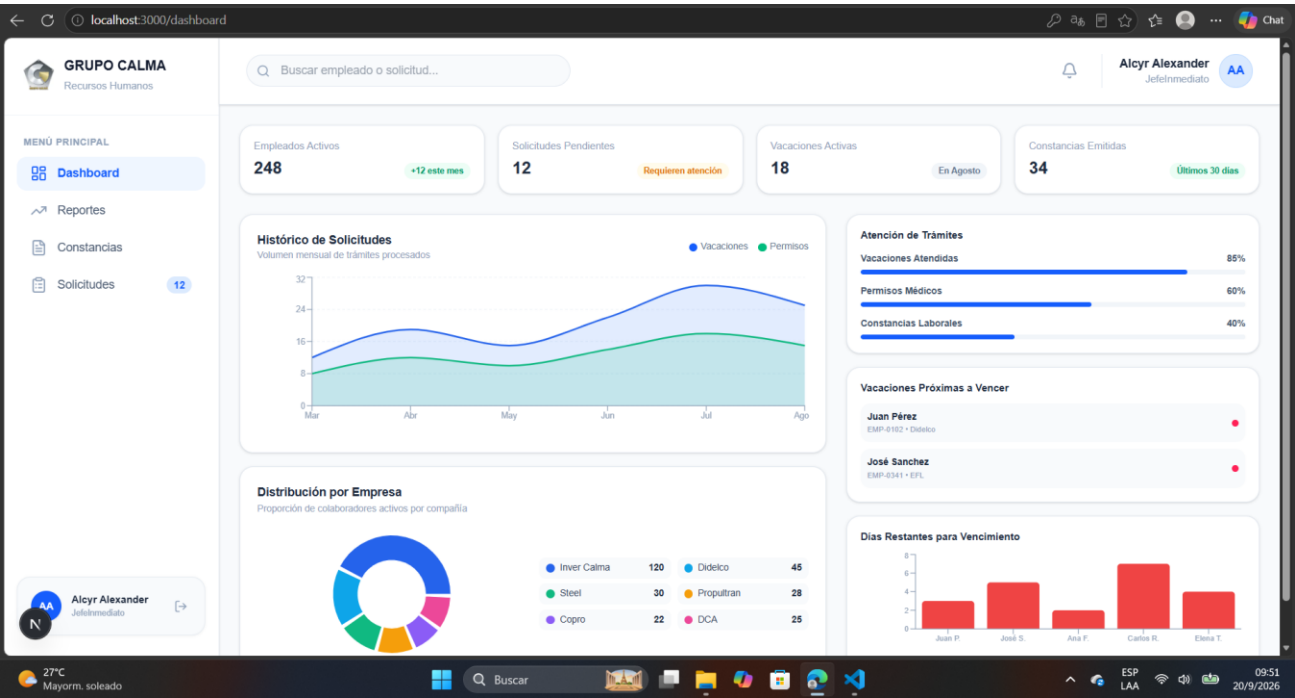


Figura 1. Sesión iniciada correctamente y acceso al dashboard con el rol Jefe Inmediato.

También se verificó el cierre de sesión para finalizar el acceso del usuario y regresar a la pantalla de autenticación.

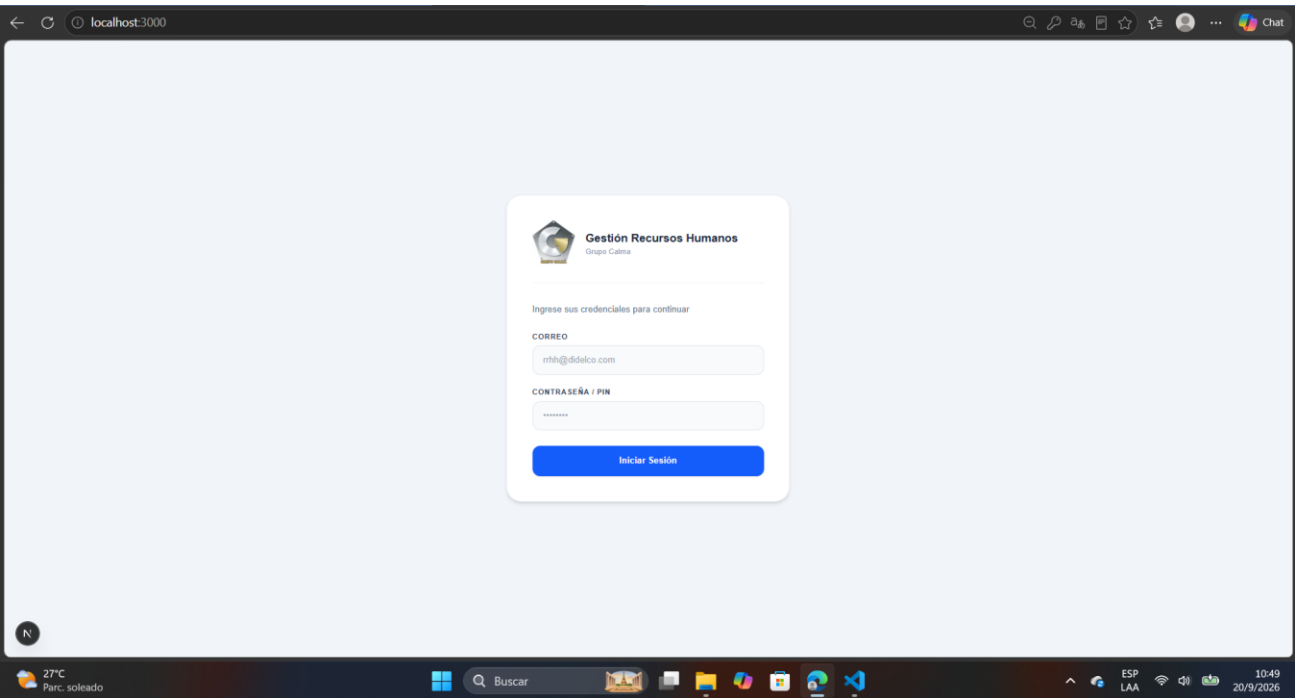


Figura 2. Prueba de cierre de sesión del sistema.

8. Dashboard

El dashboard funciona como punto central de consulta. Presenta indicadores resumidos y representaciones gráficas que permiten observar información relevante de Recursos Humanos de forma rápida.

La vista utilizada durante las pruebas muestra tarjetas de indicadores, gráficas de distribución y resúmenes de actividad, proporcionando una lectura general del estado del sistema.

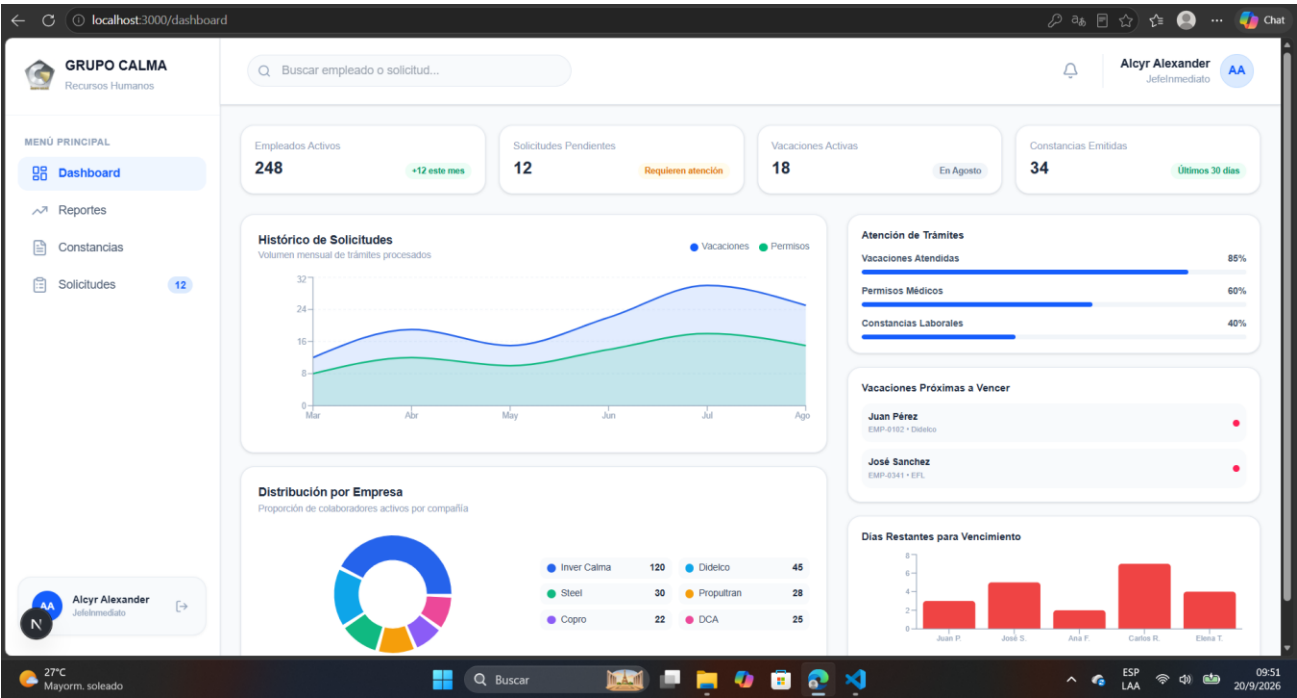


Figura 3. Dashboard principal con indicadores y gráficas de información.

9. Gestión de solicitudes

El módulo de solicitudes concentra el seguimiento de los trámites gestionados por los usuarios. La vista general permite consultar registros y distinguir su tipo y estado.

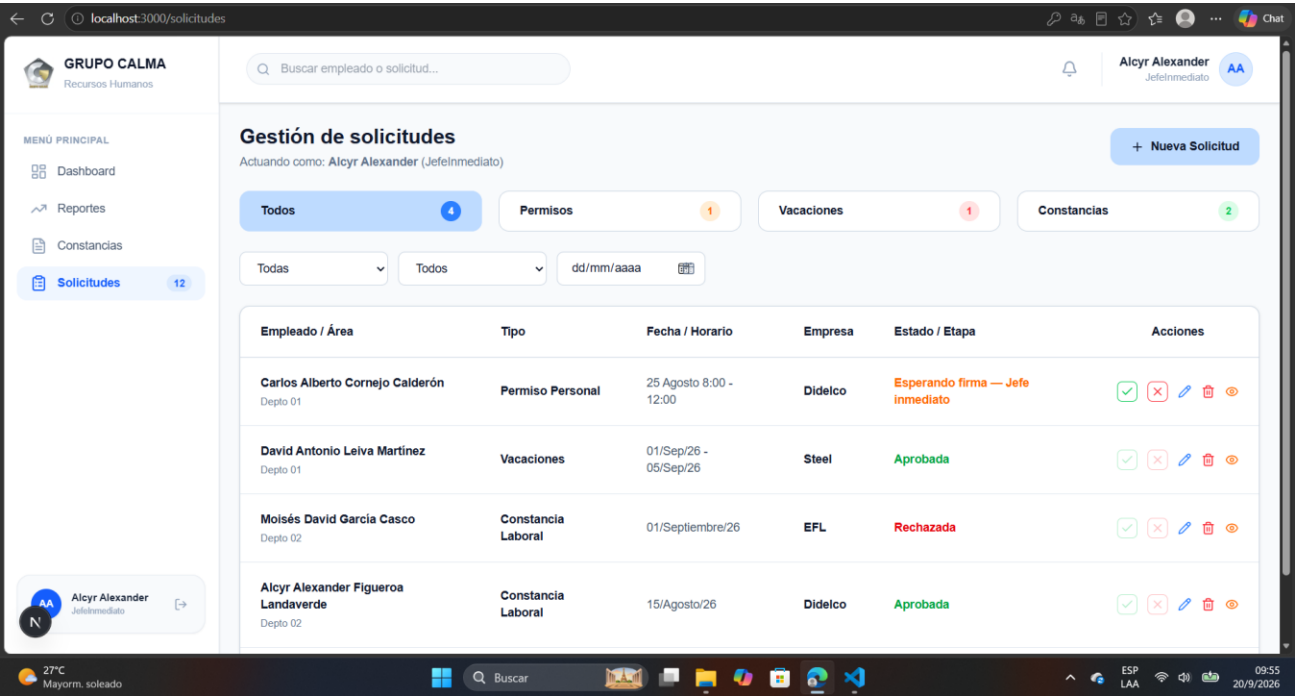


Figura 4. Vista general de la gestión de solicitudes para Jefe Inmediato.

9.1 Detalle de una solicitud

Desde la lista es posible acceder al detalle de una solicitud para revisar su información y las acciones disponibles de acuerdo con el flujo correspondiente.

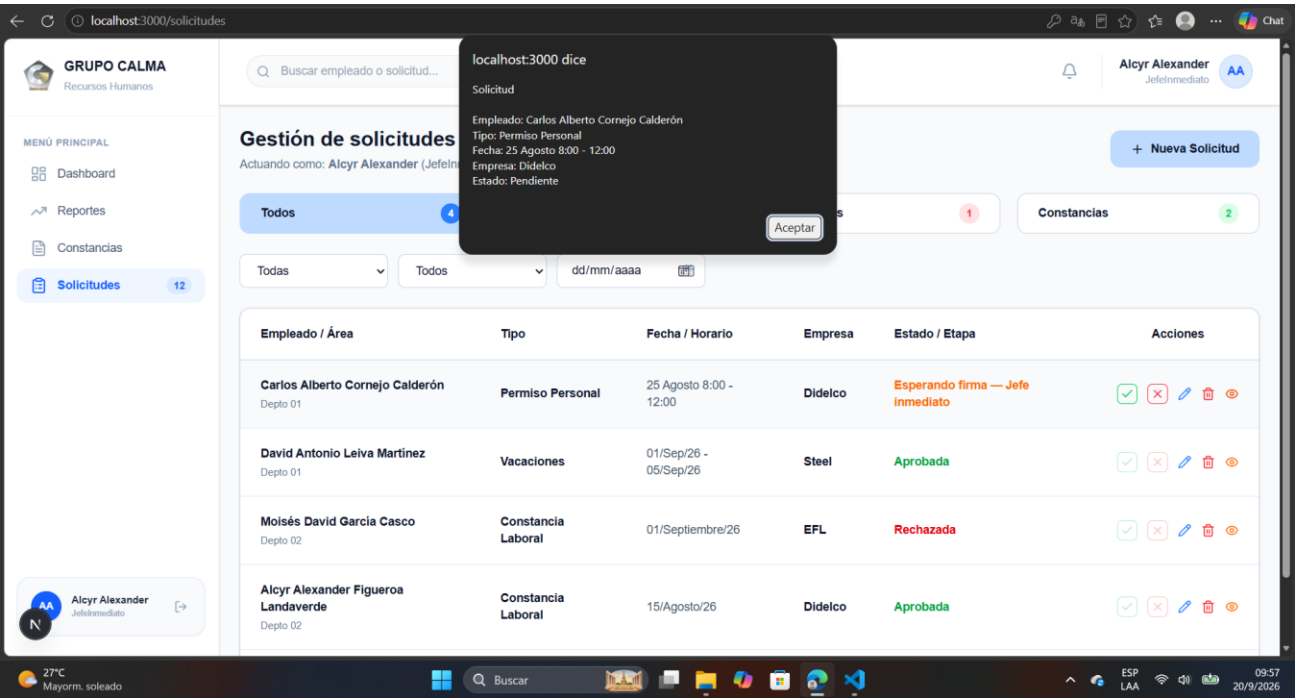


Figura 5. Visualización del detalle de una solicitud.

9.2 Filtros por tipo

El módulo permite separar la información mediante filtros de Permisos, Vacaciones y Constancias, facilitando la consulta de los registros de cada categoría.

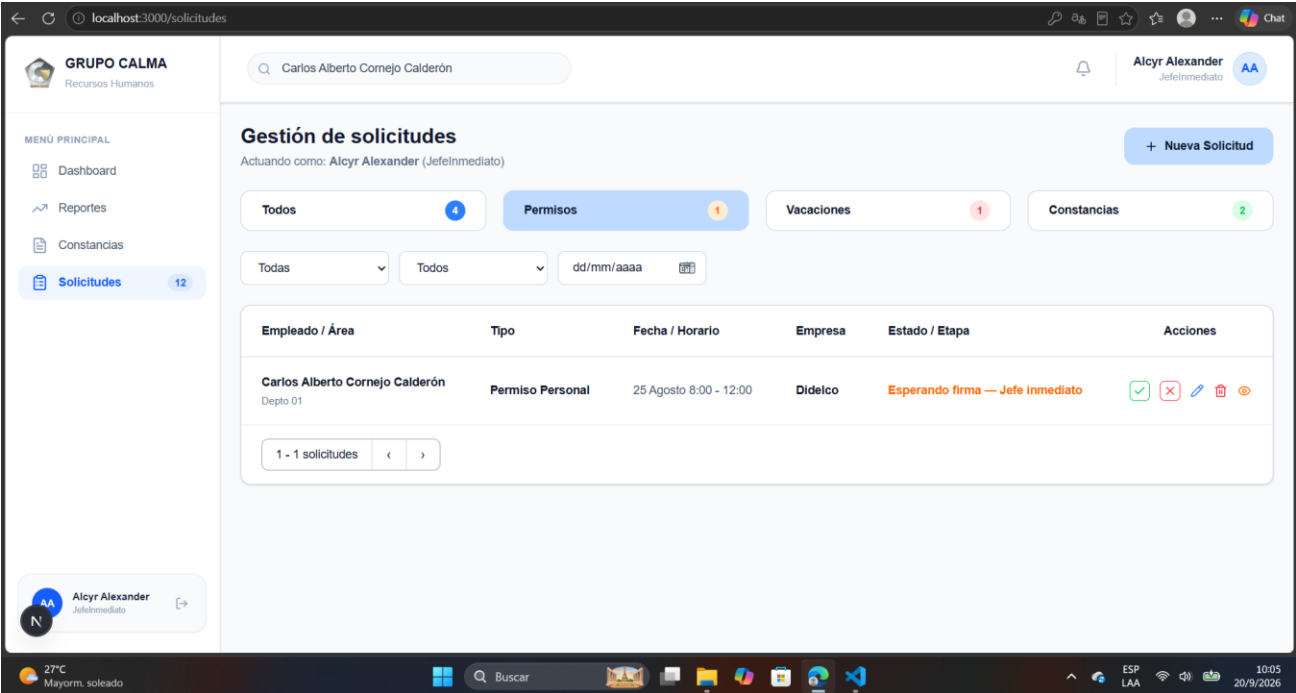


Figura 6. Filtro de solicitudes de tipo Permiso.

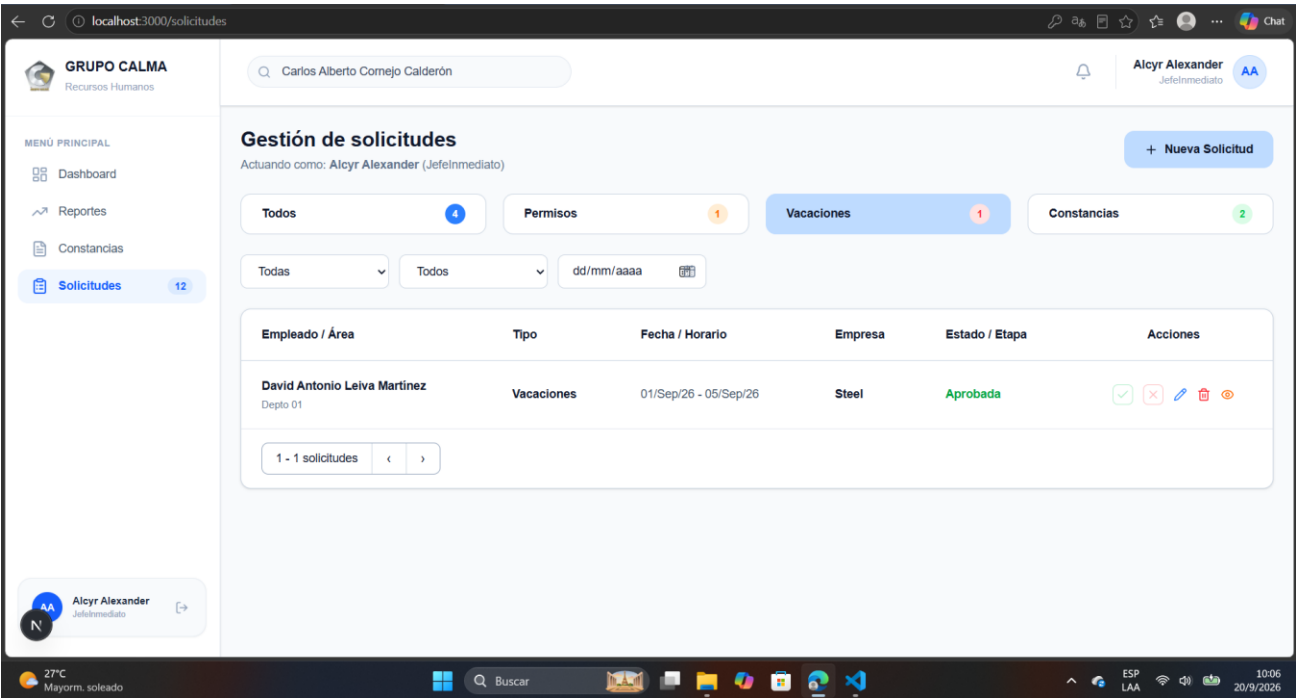


Figura 7. Filtro de solicitudes de tipo Vacaciones.

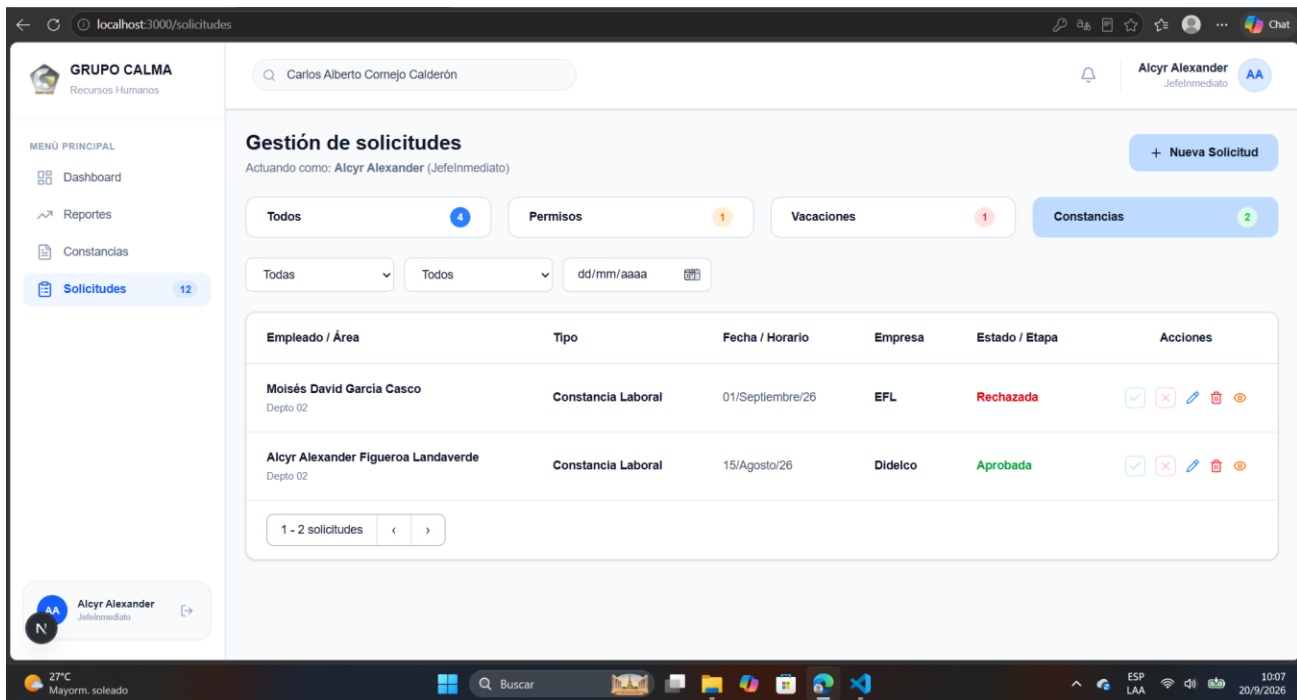


Figura 8. Filtro de solicitudes de tipo Constancia.

9.3 Registro de nueva solicitud

El sistema incorpora un formulario para registrar una nueva solicitud. El formulario reúne los datos necesarios según el tipo de trámite y permite iniciar el proceso desde la interfaz.

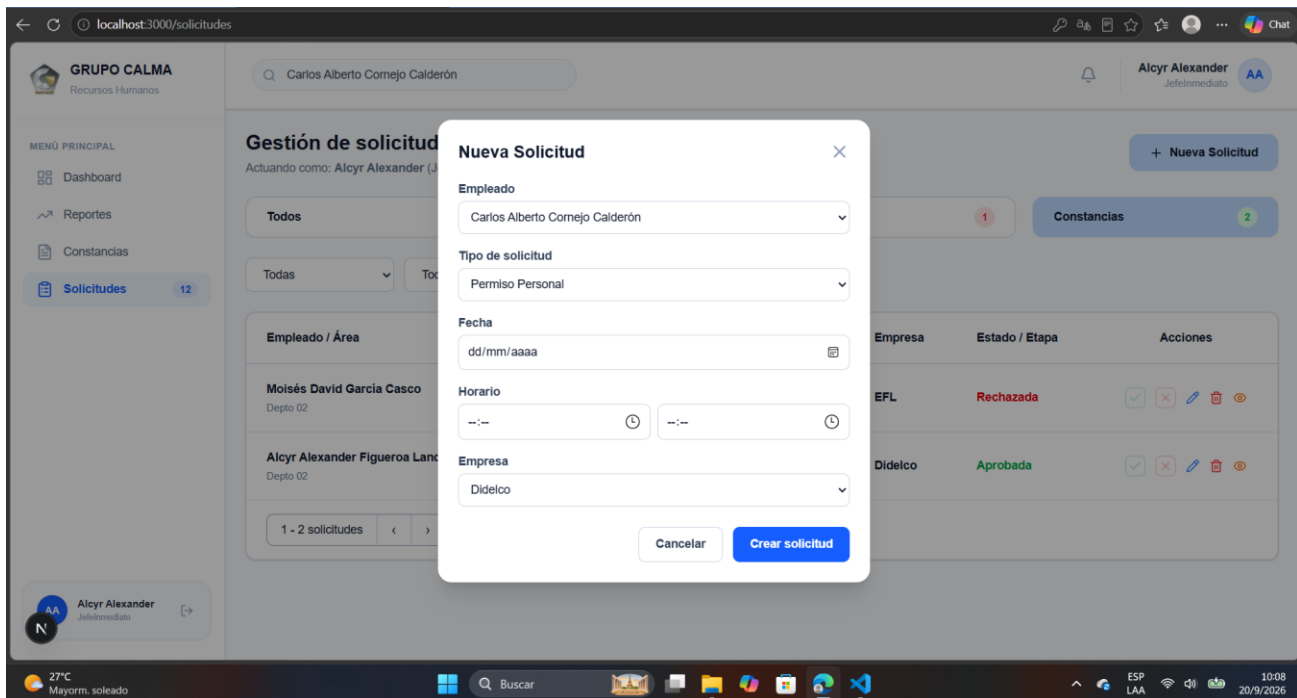


Figura 9. Formulario para registrar una nueva solicitud.

10. Flujo de aprobación

Las solicitudes manejan estados que permiten conocer su situación dentro del proceso. Entre los estados utilizados se encuentran Pendiente, Aprobada y Rechazada.

La lógica de aprobación contempla la participación de distintos roles según el tipo de trámite. Esto permite representar un flujo de revisión en el que la solicitud puede avanzar y conservar información sobre su estado.

11. Reportes

El módulo de reportes presenta una vista general de empleados y datos relacionados. Durante las pruebas se verificó la visualización de los registros y la disponibilidad de la opción de exportación.

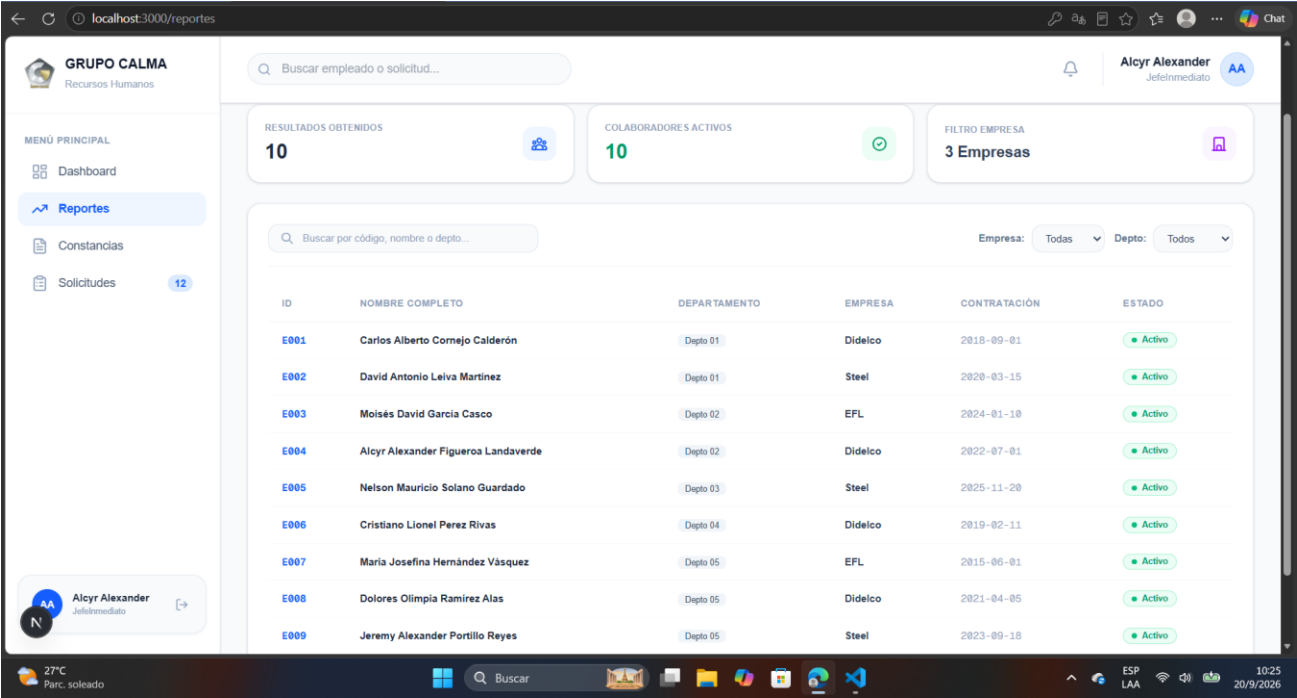


Figura 10. Vista general del módulo de reportes.

12. Constancias laborales y PDF

El proyecto incorpora la generación de constancias laborales en formato PDF. La vista permite preparar la información de la constancia y generar el documento correspondiente.

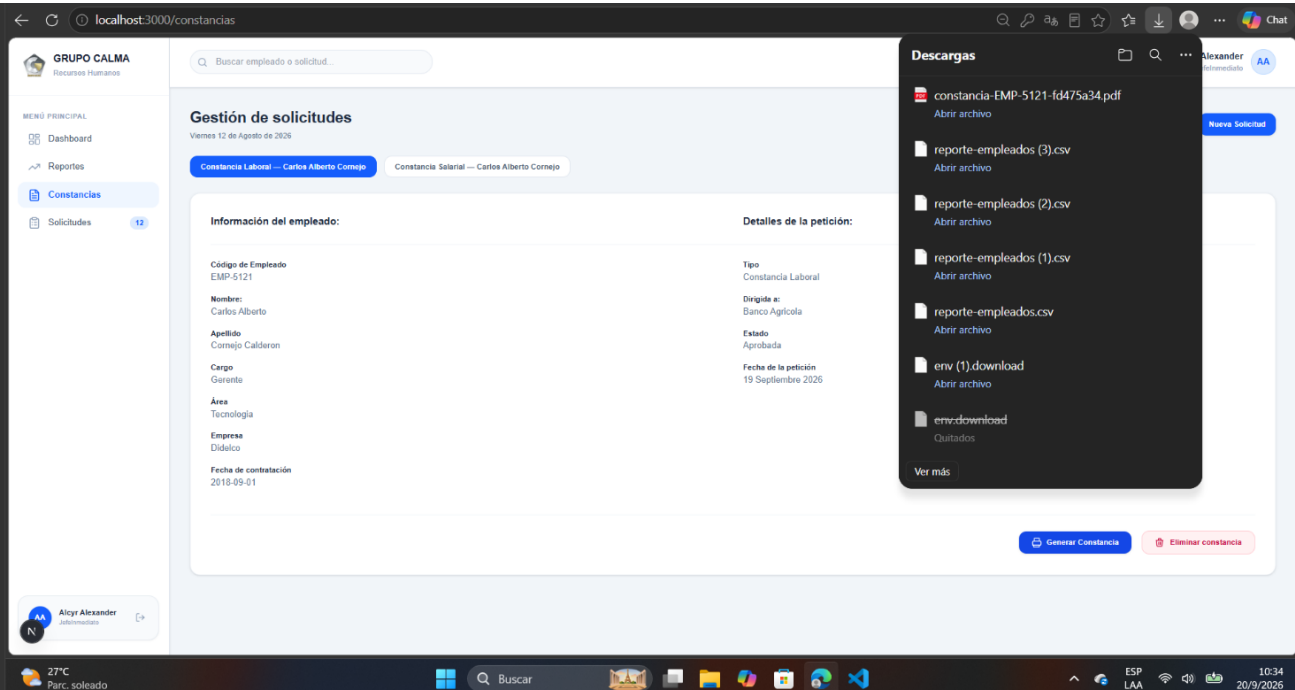


Figura 11. Vista utilizada para generar una constancia laboral.

El documento generado incorpora información del empleado y un código QR como elemento complementario de la constancia.

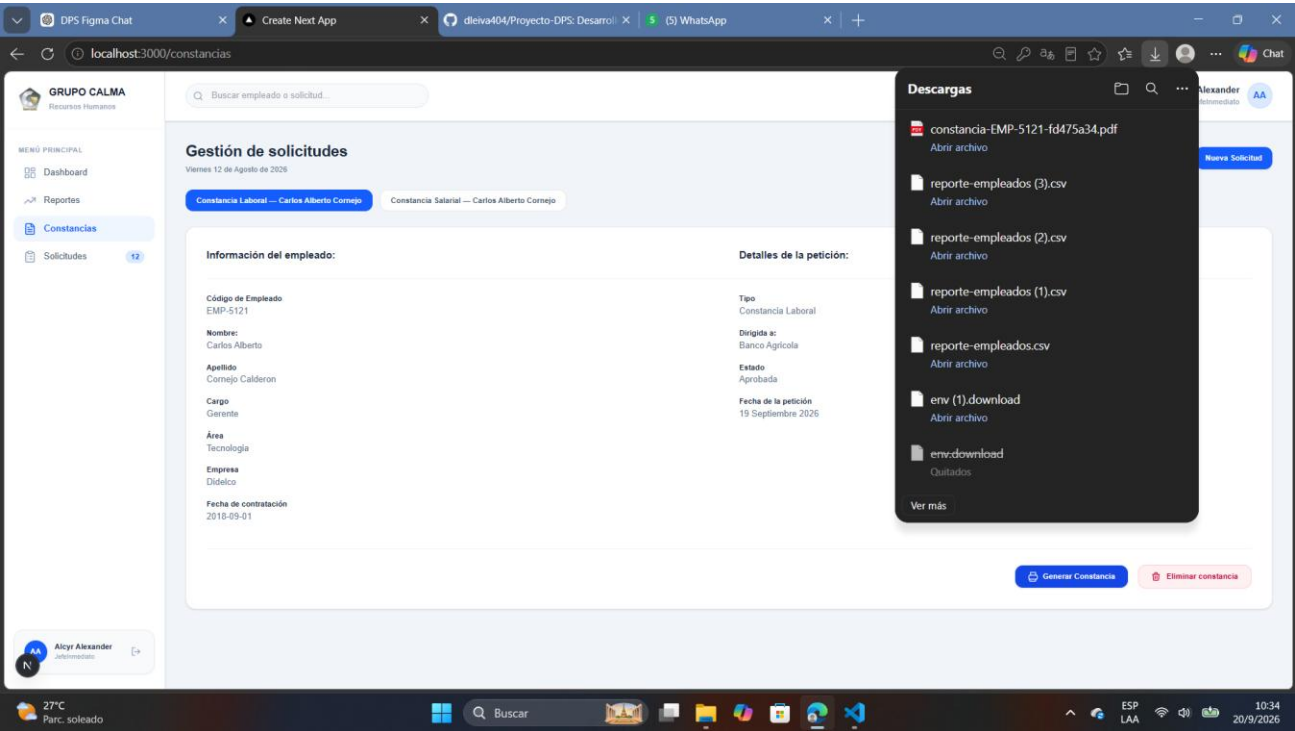


Figura 12. Prueba de constancia laboral generada por el sistema.

13. Firebase y manejo de datos

El proyecto cuenta con configuración de Firebase para autenticación y servicios relacionados con datos. La configuración sensible se mantiene mediante variables de entorno locales, evitando incluir directamente los valores en el código compartido.

```
NEXT_PUBLIC_FIREBASE_API_KEY
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN
NEXT_PUBLIC_FIREBASE_PROJECT_ID
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID
NEXT_PUBLIC_FIREBASE_APP_ID
```

Además de los servicios de Firebase, algunos módulos utilizan datos de prueba durante esta etapa para permitir el desarrollo y demostración de las interfaces mientras continúa la integración general del sistema.

14. API REST de solicitudes

Como parte de la integración cliente-servidor se implementó un endpoint REST en Next.js para trabajar con solicitudes. La ruta utilizada es:

```
/api/solicitudes
```

El endpoint utiliza Route Handlers de Next.js y contempla los métodos GET y POST.

14.1 Método GET

GET permite consultar el conjunto de solicitudes utilizado por la integración REST. La respuesta se entrega en formato JSON e incluye el resultado de la operación, el total de registros y los datos.

```
{
  "success": true,
  "total": 4,
  "data": [ ... ]
}
```

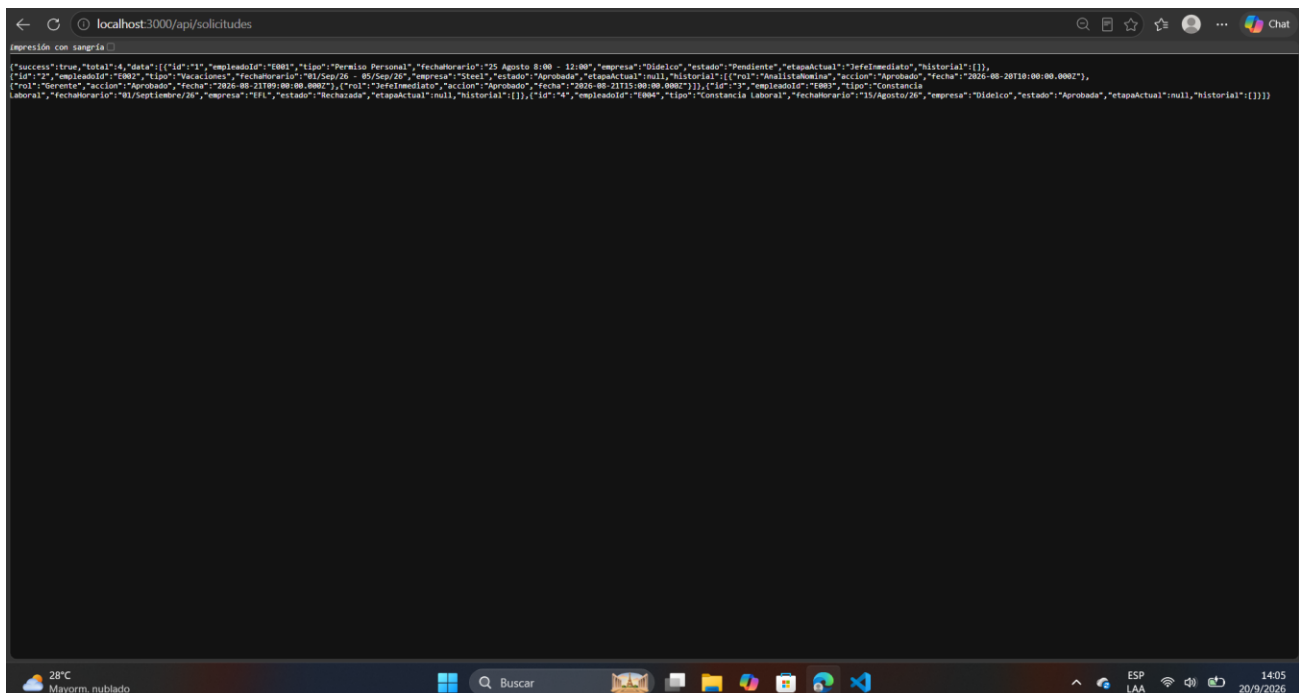
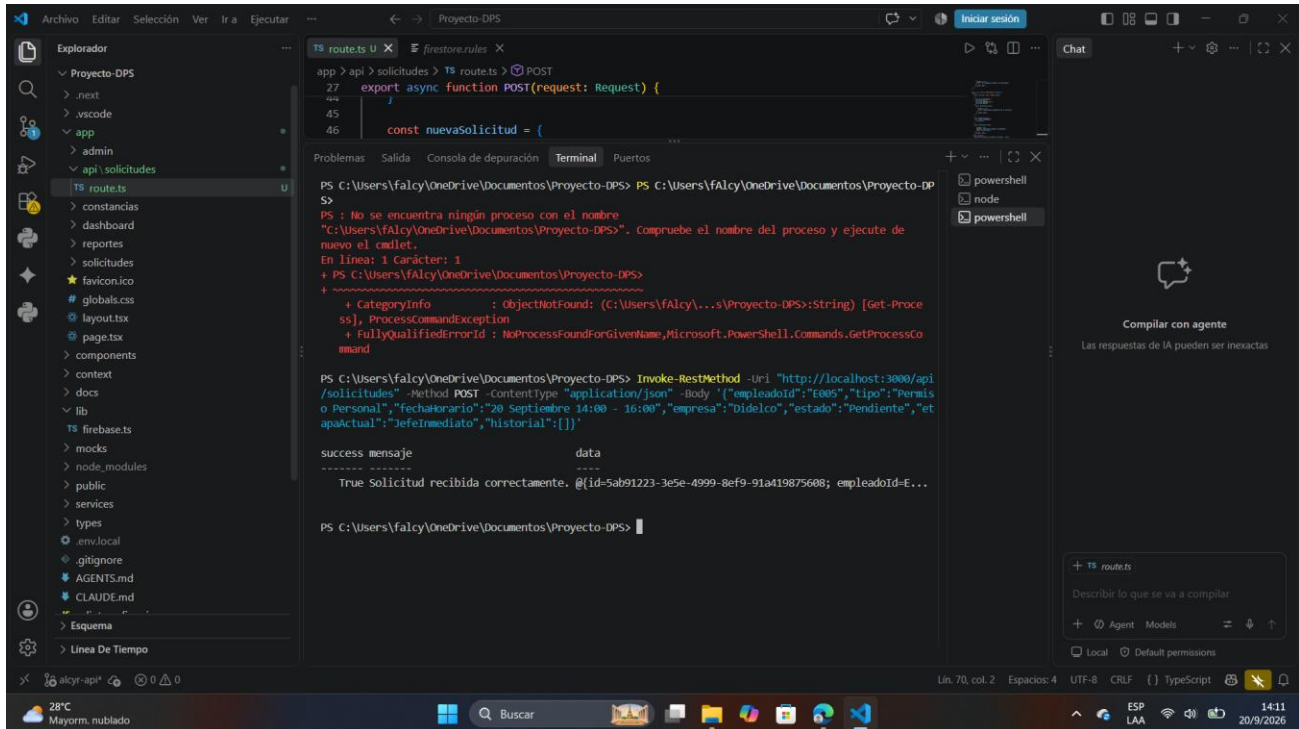


Figura 13. Prueba satisfactoria del método GET del endpoint de solicitudes.

14.2 Método POST

POST permite recibir los datos de una solicitud. Antes de responder satisfactoriamente, el endpoint verifica los campos requeridos. En la prueba realizada se obtuvo una respuesta exitosa y se generó un identificador para la solicitud recibida.

En esta integración de prueba, el POST devuelve la representación de la solicitud recibida, pero no agrega de forma permanente el nuevo registro al arreglo mock utilizado por GET.



The screenshot shows the Visual Studio Code interface with the following components:

- Explorer:** Shows the project structure for 'Proyecto-DPS', including folders like '.next', '.vscode', 'app', 'admin', 'api', 'solicitudes', and files like 'route.ts', 'constancias', 'dashboard', 'reportes', 'solicitudes', 'favicon.ico', 'globals.css', 'layout.tsx', 'page.tsx', 'components', 'context', 'docs', 'lib', 'firebase.ts', 'mocks', 'node_modules', 'public', 'services', 'types', '.env.local', '.gitignore', 'AGENTS.md', 'CLAUDE.md', 'Esquema', and 'Linea De Tiempo'.
- Editor:** Displays the 'route.ts' file with a TypeScript function for a POST endpoint. The code defines a 'nuevaSolicitud' object and uses 'Invoke-RestMethod' to send a request to 'http://localhost:3000/api/solicitudes'.
- Terminal:** Shows the execution of the test. It starts with a PowerShell prompt, followed by an error message about a missing process. Then, it shows the successful execution of the 'Invoke-RestMethod' command, returning a JSON response with a success message and a data object containing a unique ID and employee information.
- Chat:** A sidebar on the right with a 'Chat' icon and a 'Compilar con agente' button.

```
app > api > solicitudes > TS route.ts > POST
27 export async function POST(request: Request) {
45
46   const nuevaSolicitud = {
    + CategoryInfo          : ObjectNotFound: (C:\Users\FAlcy\...\Proyecto-DPS:String) [Get-Proce
    + FullyQualifiedErrorId : NoProcessFoundForGivenName,Microsoft.PowerShell.Commands.GetProcessCo
    + CommandName           : Get-Process

PS C:\Users\FAlcy\OneDrive\Documentos\Proyecto-DPS> PS C:\Users\FAlcy\OneDrive\Documentos\Proyecto-DP
S>
PS : No se encuentra ningún proceso con el nombre
"C:\Users\FAlcy\OneDrive\Documentos\Proyecto-DPS". Compruebe el nombre del proceso y ejecute de
nuevo el cmdlet.
En línea: 1 Carácter: 1
+ PS C:\Users\FAlcy\OneDrive\Documentos\Proyecto-DPS>
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (C:\Users\FAlcy\...\Proyecto-DPS:String) [Get-Proce
    + FullyQualifiedErrorId : NoProcessFoundForGivenName,Microsoft.PowerShell.Commands.GetProcessCo
    + CommandName           : Get-Process

PS C:\Users\FAlcy\OneDrive\Documentos\Proyecto-DPS> Invoke-RestMethod -uri "http://localhost:3000/api
/solicitudes" -method POST -contentType "application/json" -body '{"empleadId":"E005","tipo":"Permis
o Personal","fechaInicio":"20 Septiembre 14:00 - 16:00","empresa":"Didelco","estado":"Pendiente","et
apaActual":"JefeInmediato","historial":[]}'

success mensaje          data
-----
True Solicitud recibida correctamente. @id=5ab91223-3e5e-4999-8ef9-91a419875608; empleadoId=E...

PS C:\Users\FAlcy\OneDrive\Documentos\Proyecto-DPS>
```

Figura 14. Prueba satisfactoria del método POST del endpoint de solicitudes.

15. Consumo de API mediante Fetch

Para demostrar el consumo del endpoint desde el frontend se desarrolló una vista técnica independiente en /integracion-api. Esta vista realiza la consulta mediante Fetch API y representa la información recibida.

La integración contempla estados de carga y manejo de fallos de consulta. Se mantuvo separada del módulo principal de Solicitudes para no reemplazar el flujo de datos utilizado por ese módulo durante esta etapa.

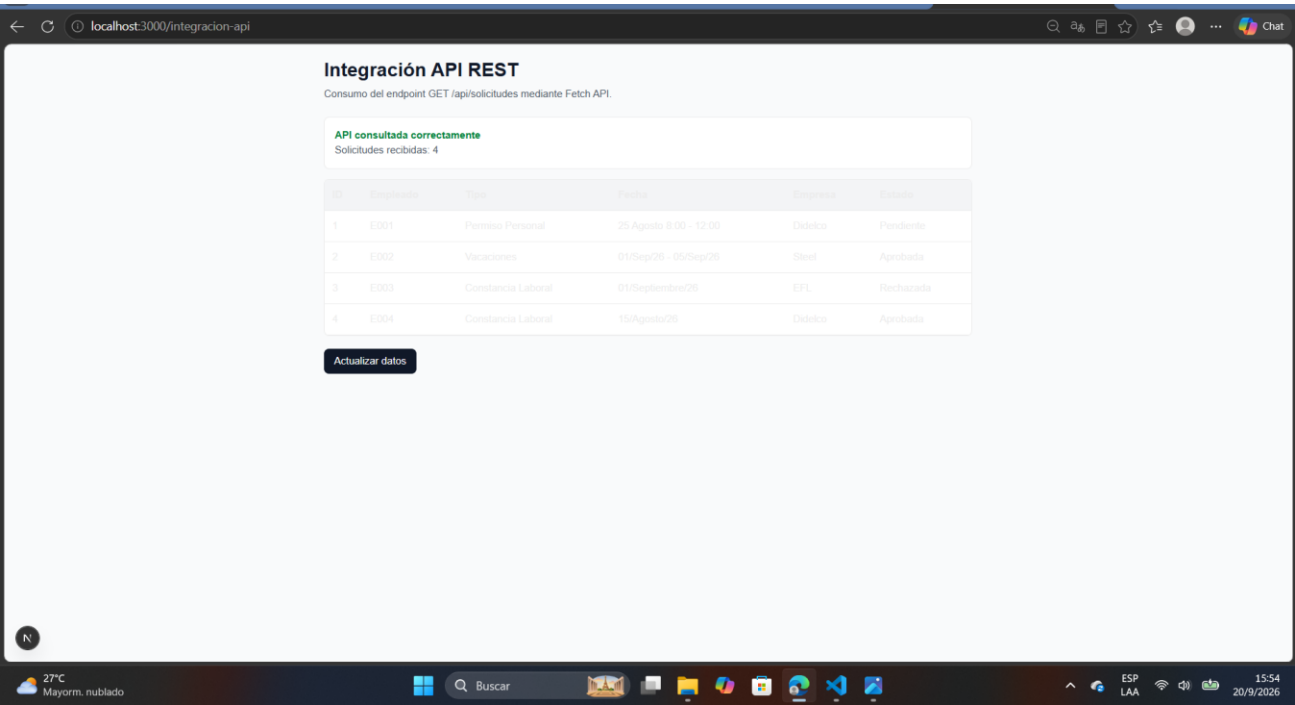


Figura 15. Estado de carga durante el consumo de la API REST.

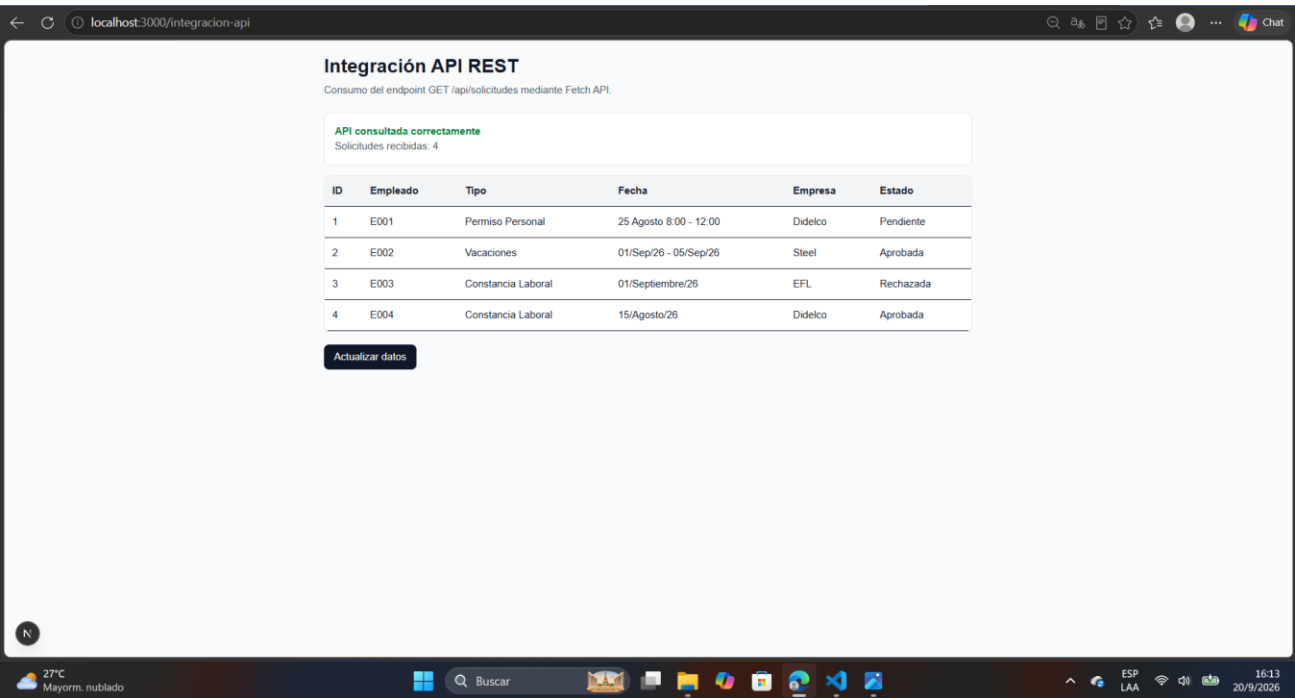


Figura 16. Resultado de la integración REST consumida desde el frontend.

16. Control de versiones y colaboración

El proyecto se administra mediante Git y GitHub. El repositorio principal es Proyecto-DPS, bajo la cuenta dleiva404. El equipo trabaja mediante ramas y commits para mantener separados los cambios de cada integrante.

Para la integración REST y su documentación se utilizó la rama alcyr-api, que fue enviada al repositorio remoto y posteriormente integrada al trabajo principal.

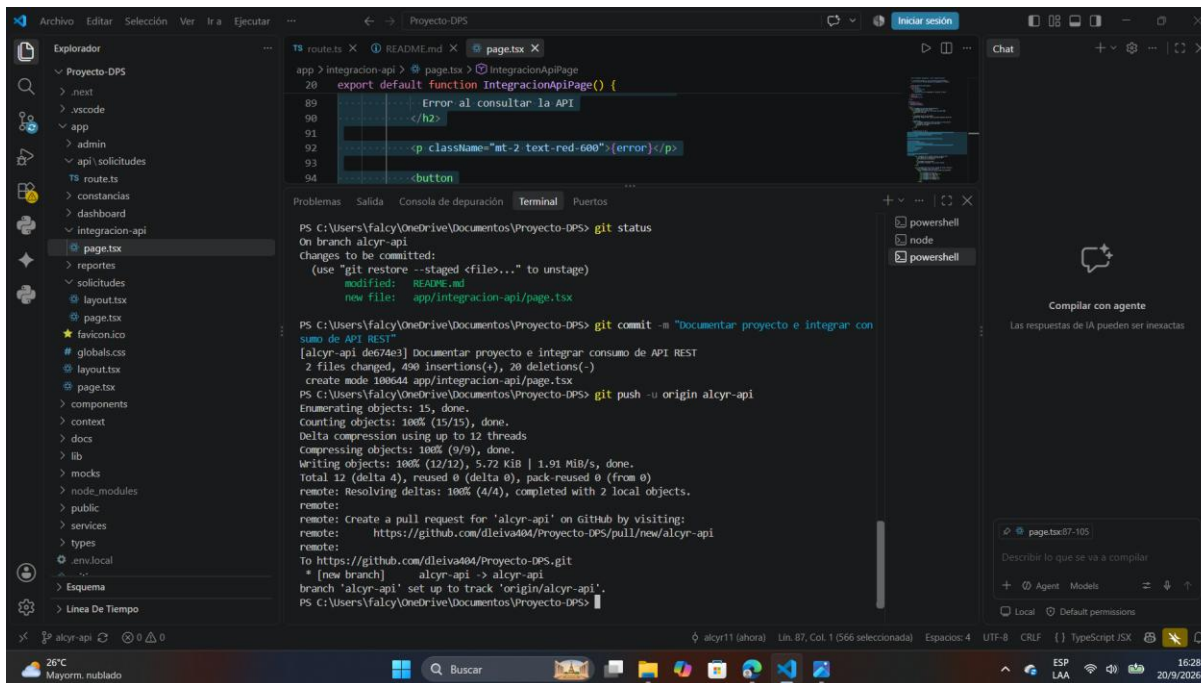


Figura 17. Envío de cambios de la rama alcyr-api al repositorio remoto.

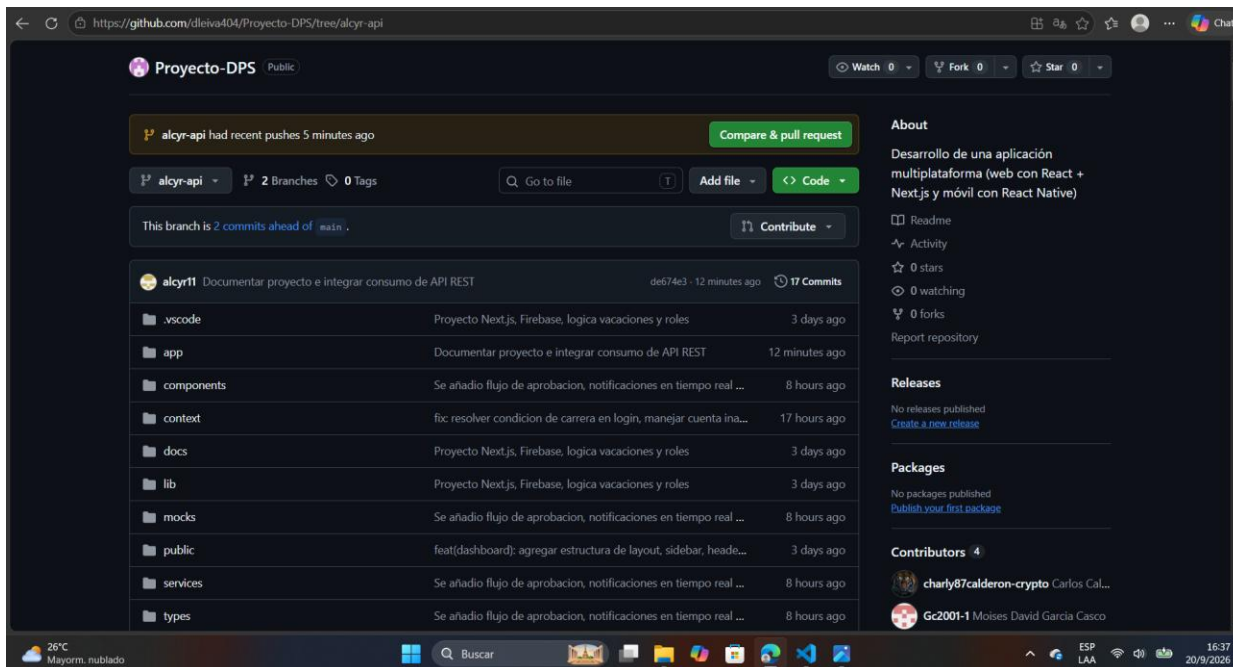


Figura 18. Rama alcyr-api disponible en el repositorio de GitHub.

El README del repositorio documenta el objetivo del sistema, tecnologías, roles, arquitectura, endpoint REST, instalación y estado del despliegue. Repositorio principal de GitHub: <https://github.com/dleiva404/Proyecto-DPS>

17. Compilación para producción

Antes del despliegue se ejecutó la compilación de producción del proyecto mediante el comando:

```
npm run build
```

El proceso finalizó correctamente. Next.js compiló el proyecto, TypeScript completó la validación y se generaron las páginas y rutas necesarias para la versión de producción.

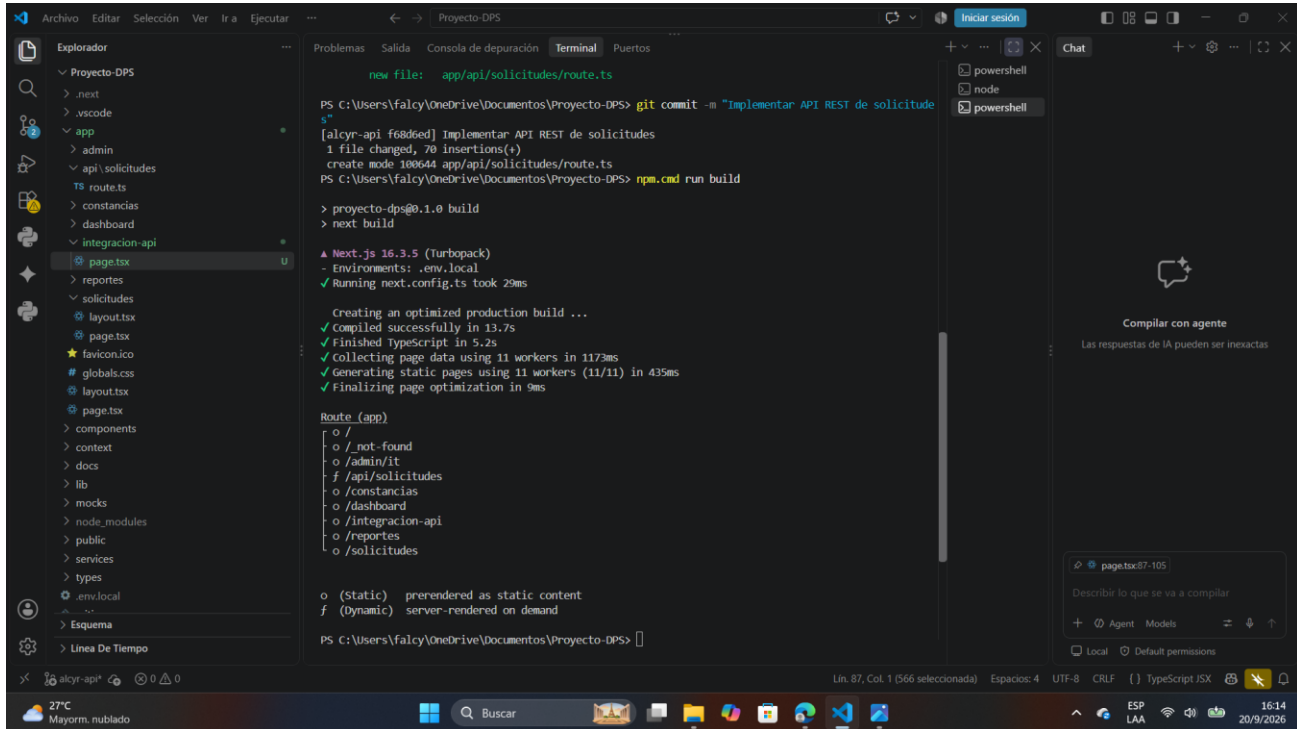


Figura 19. Compilación satisfactoria del proyecto para producción.

18. Pruebas realizadas

Las evidencias recopiladas durante la Etapa 2 permiten comprobar el funcionamiento de los principales módulos desarrollados. Las pruebas incluyeron:

- Inicio y cierre de sesión.
- Acceso al dashboard.
- Gestión y detalle de solicitudes.
- Filtros de permisos, vacaciones y constancias.
- Formulario de nueva solicitud.
- Vista general de reportes.
- Generación de constancia laboral en PDF.
- Consulta GET de la API REST.
- Envío POST a la API REST.
- Consumo de la API mediante Fetch.
- Control de versiones mediante Git y GitHub.
- Compilación de producción con Next.js.

Las capturas de errores o incidencias internas utilizadas durante el proceso de desarrollo no se incluyen como evidencia final, ya que este documento presenta las pruebas funcionales y resultados consolidados de la etapa.

19. Despliegue en Vercel

El proyecto fue desplegado satisfactoriamente en Vercel utilizando el repositorio principal de GitHub. El despliegue de producción quedó en estado Ready y permite acceder a la aplicación desde Internet sin depender del entorno local.

Repositorio de GitHub: <https://github.com/dleiva404/Proyecto-DPS>

URL pública de Vercel: <https://proyecto-dps-zg5u.vercel.app>

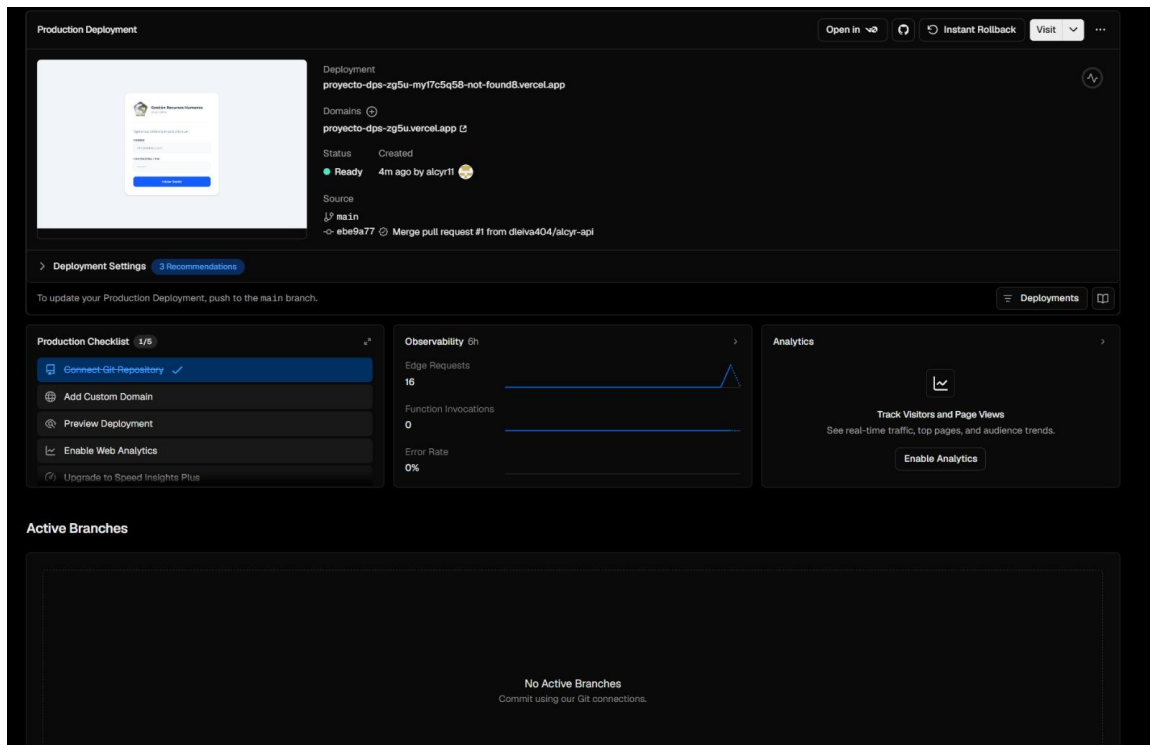


Figura 20. Despliegue de producción en Vercel con estado Ready.

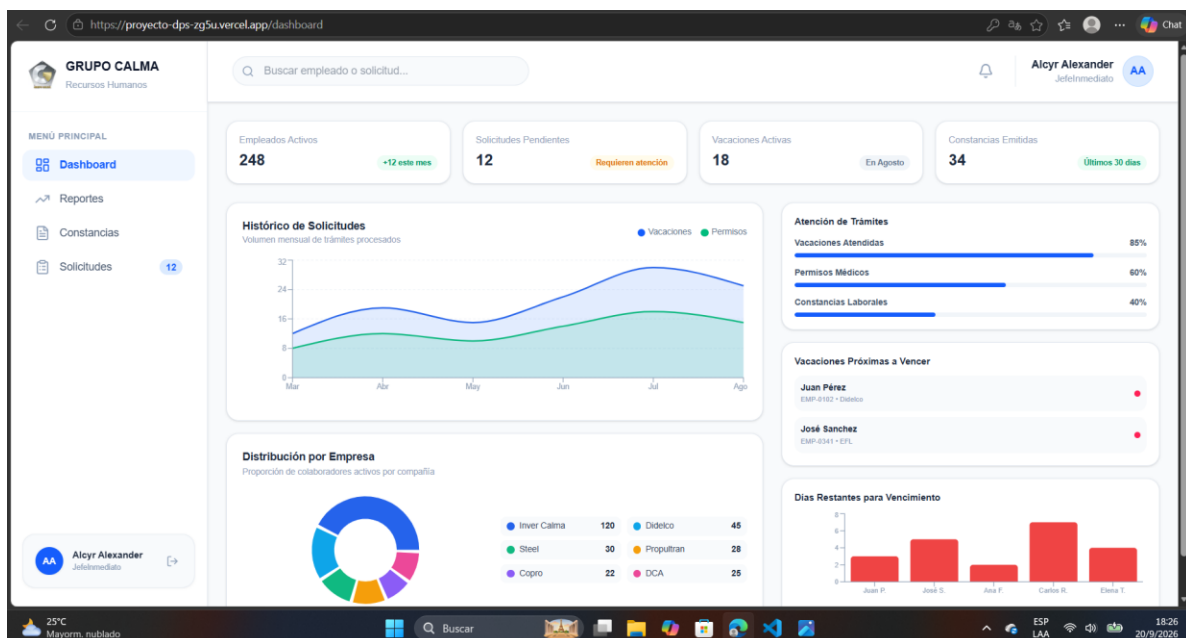


Figura 21. Aplicación pública abierta desde la URL de Vercel.

20. Resultados obtenidos

La Etapa 2 permitió establecer una base funcional del Sistema de Gestión de Recursos Humanos. Se desarrollaron los módulos principales de navegación, autenticación, dashboard, solicitudes, reportes y constancias, además de la integración de servicios y mecanismos de comunicación mediante API REST.

El uso de roles permite organizar el acceso a las funciones del sistema. La gestión de solicitudes facilita el seguimiento de permisos, vacaciones y constancias, mientras que los reportes y documentos PDF complementan las necesidades administrativas contempladas para esta fase.

La incorporación de Git y GitHub permitió mantener un proceso colaborativo de desarrollo. La compilación de producción confirmó la preparación técnica del proyecto y el despliegue final en Vercel permitió comprobar su funcionamiento mediante una dirección pública.

21. Conclusiones

El desarrollo de la Etapa 2 permitió convertir la propuesta inicial en una aplicación web funcional con una estructura preparada para continuar creciendo en las siguientes fases.

Next.js, React y TypeScript proporcionaron una base organizada para la interfaz y la lógica del sistema, mientras que Firebase aporta servicios para autenticación y manejo de datos. La API REST implementada permitió demostrar el intercambio de información mediante solicitudes HTTP y su consumo desde el frontend.

La distribución del trabajo mediante GitHub permitió integrar aportes de diferentes integrantes manteniendo control de versiones. Finalmente, la compilación de producción y el despliegue en Vercel fueron completados satisfactoriamente, dejando disponible una versión pública y funcional del sistema.

22. Integrantes del equipo

Integrante
Carlos Alberto Cornejo Calderón
David Antonio Leiva Martínez
Moisés David García Casco
Alcyr Alexander Figueroa Landaverde
Nelson Mauricio Solano Guardado

23. Estado de cierre de la documentación

Elemento	Estado
Documentación general del sistema	Completa
Evidencias funcionales recibidas	Integradas
API REST y consumo Fetch	Documentado
GitHub y compilación	Documentado
Capturas de errores	Excluidas del documento final
Material ajeno a Recursos Humanos	Excluido
URL pública de Vercel	Integrada
Evidencia final de Vercel	Integrada